

5.1-TÉCNICAS DE PROGRAMACIÓN SEGURA

Contenido

- TECNICAS DE PROGRAMACIÓN SEGURA..... 1
 - 1. Introducción a la Seguridad en la Programación 2
 - 1. Validar siempre las entradas 2
 - 2. Control de acceso fuerte 2
 - 3. Gestión segura de sesiones 3
 - 2. Principios y Prácticas Criptográficas..... 4
 - 1. Cifrado simétrico 4
 - 2. Cifrado asimétrico 4
 - 3. Sistemas híbridos 4
 - 3. Funciones Hash y Firmas Digitales 5
 - 4. Protocolos y Seguridad en la Transmisión 5
 - TLS / SSL (HTTPS) 5
 - Certificados y PKI (Infraestructura de Clave Pública) 5
 - Clave cuántica (QKD) 5
 - 5. Técnicas para Asegurar Información en Tránsito..... 6
 - Confidencialidad..... 6
 - Integridad 6
 - Autenticación 6
 - Prevención de ataques Main-in-the-Middle (MITM) 6

1. Introducción a la Seguridad en la Programación

La seguridad de software significa diseñar, escribir y mantener código que proteja la información y los recursos frente a ataques, errores o manipulaciones maliciosas. Los programas inseguros pueden permitir intrusiones, robo de datos o interrupciones en el servicio.

Principios básicos de programación segura:

Estos principios ayudan a reducir errores vulnerables y defender el software.

1. Validar siempre las entradas

1. Nunca confiar en datos del usuario.
2. Validar tipo, longitud, formato y contenido.
3. Evitar inyección de código (SQL,XSS, etc.).

Inyección de código SQL (SQL Injection)

Ocurre cuando una aplicación construye consultas SQL usando directamente datos introducidos por el usuario, sin validarlos ni protegerlos correctamente.

El atacante introduce código SQL malicioso en campos como:

- Usuario.
- Contraseña.
- Formularios.
- Parámetros de URL.

El código se ejecuta en la base de datos como si fuera legítimo.

2. Control de acceso fuerte

1. *Políticas de contraseñas robustas*

- **Longitud mínima:** Que la contraseña tenga una longitud mínima recomendada entre 12 y 16 caracteres, priorizando longitud sobre complejidad extrema.
- **Utilizar diversidad de caracteres:** Mezclando tanto letras en mayúscula, minúscula, números y símbolos a la vez.
- **No reutilizarla:** No debe de usarse en distintos servicios.
- **No basada en datos personales:** No utilizar datos personales como el nombre, fecha de nacimiento, DNI... para la contraseña.

2. *Autenticación multifactor (MFA)*

La autenticación multifactor (MFA) añade más de una prueba de identidad al inicio de sesión.

- **Algo que sabes:** Como una contraseña o un PIN.
- **Algo que tienes:** Elemento no necesariamente material como un móvil, app autenticadora, token, SMS...
- **Algo que eres:** Único para poder identificarse como tu huella dactilar, reconocimiento facial, el iris...

Las ventajas de la autenticación multifactor (MFA) son, entre otras, que a pesar de que te puedan robar la contraseña no pueden acceder y que reduce drásticamente los ataques por phishing.

3. *Separar roles y permisos*

La separación de roles y permisos consiste en no dar a todos los usuarios los mismos privilegios. Sino asignarlos según su función. En otras palabras, ocultar u bloquear todas las funciones a todos los usuarios e ir habilitando solo las necesarias. A esto se le llama el principio del menor privilegio.

- **Error común:** Un único usuario tiene permisos totales para todo.
- **Beneficios:** Reduce el impacto de los ataques, mejora la trazabilidad y evita errores humanos graves.

3. Gestión segura de sesiones

1. *Tokens de sesión con expiración*

Un token de sesión es un identificador único y aleatorio que representa una sesión autenticada.

¿Cómo funciona?

- Usuario se autentica.
- El servidor genera un token.
- El token se envía al cliente.
- El cliente lo envía en cada petición.

Características de un token seguro

- Aleatorio.
- Imposible de adivinar.
- Con caducidad.
- Revocable.

La mayor ventaja de los tokens de sesión es el evitar el uso el usuario y contraseña en cada petición.

2. *Marcar cookies como Secure y HttpOnly*

Las cookies pueden almacenar tokens de sesión, por lo que es necesario que se protejan.

- **Secure:** La cookie solo se puede enviar por Https.
- **HttpOnly:** No se puede acceder a la información de la cookie desde JavaScript.

3. *No reinventar la criptografía*

4. Usar bibliotecas probadas (OpenSSL, NaCl)

Lo primordial es no diseñar algoritmos propios ya que diseñar criptografía propia es una de las peores prácticas de seguridad. El principio básico es “La criptografía debe ser pública y revisada.”.

- **Problemas:** Los problemas que puede acarrear el diseñar criptografía propia son los siguientes:
 - Errores matemáticos.
 - Falta de revisión por expertos.
 - Vulnerabilidades ocultas.
 - Sensación falsa de seguridad.
- **Solución correcta:** Usar algoritmos estándar como AES, RSA, ECC, SHA-256... e implementarlos mediante librerías probadas.

5. Encriptar datos sensibles

Siempre cifrar datos personales o financieros.

2. Principios y Prácticas Criptográficas

La criptografía es un conjunto de técnicas matemáticas que protegen la información garantizando los siguientes puntos:

- **Confidencialidad:** La cookie solo se puede enviar por Https.
- **Integridad:** No se puede acceder a la información de la cookie desde JavaScript.
- **Autenticación:** La cookie solo se puede enviar por Https.
- **No repudio:** No se puede acceder a la información de la cookie desde JavaScript.

Tipos de cifrado:

1. Cifrado simétrico

- Usa la misma clave para cifrar y descifrar.
- Rápido y eficiente, ideal para grandes volúmenes de datos.
- Ej.: AES, DES.

2. Cifrado asimétrico

- Usa un par de claves: pública para cifrar y privada para descifrar.
- Ideal para intercambio de claves y firmas digitales.
- Ej.: RSA, ECC.

3. Sistemas híbridos

- Combinan cifrado simétrico y asimétrico: Asimétrico para intercambiar una clave simétrica luego usar ésta última para cifrar mensajes.

3. Funciones Hash y Firmas Digitales

Funciones Hash: Estas funciones se encargan de transformar datos de longitud variable en un valor fijo (huella). Estas huellas se usan para comprobar la integridad ya que, si cambia aún que sea un bit, el hash cambia.

Firmas digitales: Las firmas digitales se basan en criptografía asimétrica. Estas, permiten comprobar que un mensaje proviene de un remitente específico y que no se ha modificado. Una firma digital permite garantizar la autenticidad, la integridad y el no repudio. Que el autor no puede negar haber firmado el documento.

¿Cómo funciona?

- Se calcula el hash del documento.
- El hash se cifra con clave privada del emisor.
- El receptor descifra con la clave pública.
- Si el hash coincide el documento es válido.

Usos comunes

- Contratos electrónicos.
- Correos firmados.
- Certificados SSL / TLS.
- Software formado.

4. Protocolos y Seguridad en la Transmisión

Asegurar la información transmitida implica protegerla frente a escuchas y manipulación.

Protocolos seguros:

TLS / SSL (HTTPS)

- Garantiza que la comunicación web esté cifrada.
- Usa certificados digitales para validar la identidad.

Certificados y PKI (Infraestructura de Clave Pública)

- Son Autoridades de Certificación (CA) emiten certificados X.509, que sirven para autenticar servidores y usuarios.

Clave cuántica (QKD)

- Usa principios de mecánica cuántica para generar claves secretas.
- Cualquier intento de espionaje se detecta automáticamente.

5. Técnicas para Asegurar Información en Tránsito

Al transmitir datos entre sistemas, hay que proteger lo siguiente:

Confidencialidad

- Cifrar todo lo que se envía (TLS, VPNs, cifrado de extremo a extremo).

Integridad

- Hash + firmas digitales para verificar que no hubo alteración.

Autenticación

- Certificados, tokens firmados, autenticación multifactor.

Prevención de ataques Main-in-the-Middle (MITM)

- Validar certificados y claves de seguridad en canales (HTTPS, SSH).
-